RouteWise: Picking Shipment Routes When Cheap, Fast and Safe Disagree

Mahesh Kumar Pandey · maheshkumarpandey17@gmail.com · Aiqon AI Internship take-home
Code: github.com/Mp17082005/routewise · Live demo: routewise-123.streamlit.app

Summary

I built a tool that takes one shipment and its 3–5 possible routes, recommends a route, explains the choice in plain English, and says how confident it is. It works in four steps. First it removes any route that breaks a hard rule, such as missing a hard deadline. Next it removes any route that another route beats on every measure. It then scores the routes that are left, using weights that depend on how urgent the shipment is. Finally it tests whether the answer changes when those weights are nudged.

On 40 test shipments, it missed deadlines on **6.0%** of shipments. Always picking the cheapest route missed **22.7%**. The price of that is a **53% higher average freight cost than always-cheapest**. Of all the strategies I compared, RouteWise made the most balanced choices. I also report the places where it does *not* win.

1. What I mean by the "best" route

There is no best route in general. A cheap route is usually slow. A fast route is usually expensive or passes through a risky area such as the Suez Canal. A shipment of urgent medicine and a shipment of steel should not be routed the same way. So I started by writing down exactly what "best" means:

The best route is the one that (1) obeys the shipment's hard rules on deadline, budget and risk limit, (2) is not beaten on every measure by another route, and (3) among what is left, gives the best balance of cost, travel time, chance of being late and risk, weighted by what this shipment cares about most.

I kept hard rules separate from the scoring on purpose. If a route will miss a hard deadline, it should be ruled out, not just given a lower score. Otherwise a big enough discount could "buy" a late delivery, and no operator would accept that.

2. One shipment, start to finish

Shipment **S004** goes from Shanghai to Rotterdam. It is a *budget* shipment, so cost matters most (60% of the weight). It also has a **hard deadline of 36 days**. There are five options:

Route	Path	Cost	Avg days	Bad-case days*	Result
R-1	**Shanghai → Rotterdam (direct)**	**$2,924**	**28.8**	**31.3**	**Recommended**
R-2	via Singapore, Mumbai, Colombo, Suez	$1,614	46.2	49.5	Too late
R-3	via Los Angeles, New York (rail+sea)	$2,311	39.6	41.9	Too late
R-4	via Los Angeles, New York (rail+sea)	$2,282	39.6	41.5	Too late
R-5	via Busan, Los Angeles, New York	$1,796	44.2	45.9	Too late

* Bad-case days: 9 out of 10 simulated trips arrive by this day (the "P90").

A plain "pick the cheapest" rule chooses R-2, which is about 10 days late on average and 13 days late in a bad case. RouteWise rules it out and recommends R-1. It costs $1,310 more, but its chance of missing the deadline drops from 100% to 1%. The explanation the tool prints says exactly that, using these numbers, and lists why each other route was dropped. When I randomly changed the weights 500 times, R-1 still won every time, so the tool marks the answer **High confidence**.

3. How it works, and why I built it this way

Step	What it does	Why I chose this
1. Simulate delays	Runs each route 1,000 times with random delays to get a typical arrival time, a bad-case arrival time (the 90th percentile, P90) and the chance of being late.	An average hides risk. A route that is fine on average but late 4 times in 10 is not safe.
2. Apply hard rules	Drops routes whose **bad-case** time misses a hard deadline, that exceed the budget, or that are too risky for fragile, perishable or high-value cargo.	Some rules are not negotiable. If no route passes, it still returns the least-bad route, clearly marked NON-COMPLIANT.
3. Remove clearly worse routes	Drops any route that another route beats on cost, time, lateness and risk all at once (called a "Pareto" filter).	Prevents obviously bad picks, and lets the explanation say *which* route beat it.
4. Score what is left	Rescales each measure from 0 (best option here) to 1 (worst option here), then takes a weighted sum. The lowest score wins.	Simple and transparent. An operator can see exactly why one route beat another.
5. Explain	Fills in fixed sentence templates with the real numbers: why it won, what was traded off, and how it compares with the cheapest and fastest routes.	No AI text generator, so it cannot invent numbers. A test checks every figure against the data.
6. Check confidence	Nudges the weights randomly by up to ±20%, 500 times, and counts how often the same route still wins.	The weights are my judgement. This shows whether the answer depends on them.

The weights. Each shipment gets a profile based on its priority. For *urgent*, time counts most (40%). For *standard*, the weights are balanced. For *budget*, cost counts most (60%). Fragile, perishable or high-value cargo uses a *critical* profile where risk counts most (35%). These are my own sensible defaults, not values learned from data, and anyone can change them in a settings file without touching the code.

Things I chose not to use. *Machine learning:* there is no record of real outcomes to learn from, and a model would be harder to explain. *An optimisation solver:* with only 3–5 routes per shipment, checking every route already gives the exact answer.

4. The test data

No public dataset fits this problem, so I generated one, as the brief suggests. It has 14 real ports, including Mumbai, Singapore, Suez, Cape Town, Rotterdam, Shanghai and Los Angeles, which form **40 shipments with 163 routes**. I built the trade-offs in deliberately. Routes with more stops are cheaper but slower. Routes through Suez are fast but risky. The route around Africa is safe but slow. Air routes are fast and expensive. Within a shipment, cost and time move in opposite directions (correlation –0.37). I also planted hard cases to test the tool: 4 "traps" where the cheapest route misses a hard deadline, 4 near-ties, and 2 shipments where no route can meet every rule. The data is the same on every run (fixed random seed).

5. Results

There is no answer key for a decision like this, so I compared RouteWise with simpler strategies. To make the comparison fair, I also measured **expected landed cost**: freight, plus the cost of goods sitting in transit, plus a late penalty. RouteWise does not optimise this number, so it is an independent check. **Balance** measures how badly a choice does on its *worst* measure, compared with the other routes that obey the rules. Lower is better.

Strategy	Avg cost	Avg days	Late rate	Balance	Landed cost	Broke a rule
RouteWise	**$15,596**	**26.5**	**6.0%**	**0.62**	**$23,004**	**5%**
RouteWise without hard rules	$13,708	28.6	11.2%	0.68	$20,840	15%
Equal weights, no rules	$16,795	23.4	8.5%	3.62	$22,674	32%
Always fastest	$21,902	18.5	5.7%	3.69	$26,120	32%
Always safest	$17,731	27.3	26.4%	8.67	$26,924	28%
Random pick	$15,117	30.0	23.6%	11.00	$23,349	40%
Always cheapest	$10,188	32.6	22.7%	18.19	$18,472	55%

Table: all strategies on the same 40 shipments. RouteWise's 5% is exactly the 2 shipments where no route can meet every rule. Both are flagged NON-COMPLIANT, not hidden.

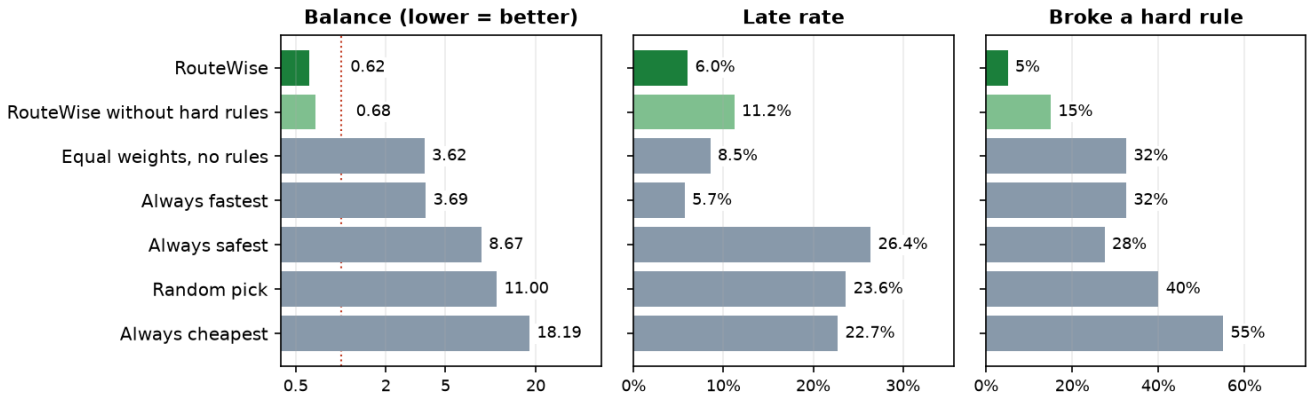


Chart: the same comparison; lower is better in every panel. The dotted line marks a balance score of 1.0, meaning "as bad as the worst allowed route on some measure".

What the results show

- **Most balanced.** RouteWise's balance score is 0.62, against 3.69 for always-fastest and 18.19 for always-cheapest. It is rarely the best on any single measure, but it never makes a one-sided choice.
- **Priorities change the answer as they should.** When I forced every shipment through each profile, urgent shipments averaged 23.5 days and $19,123. Budget shipments averaged 30.9 days and $13,671.
- **The answers are stable.** With the weights randomly nudged, the same route still won **97%** of the time. Of the 40 decisions, 31 were High confidence, 8 Medium and 1 Low. The Low one is shown as "close call, review manually". The tool also reports *switching points*, for example "the answer changes if the cost weight rises above 0.46".
- **Where it does not win.** It costs 53% more than always-cheapest. On landed cost, always-cheapest ($18,472) beats it ($23,004). That is mostly the price of the hard rules: they rule out cheap routes that are too late or too risky, and the landed-cost formula gives no value to staying within a risk limit.
- **Bad input data is the biggest weakness.** When I added 15% random error to the route estimates, the recommendation changed for 30% of shipments. In real use, accurate estimates matter more than fine-tuning the weights.

6. Problems I found while building it

- **Deadlines that could not be met.** At first I set deadlines from the *average* trip time but checked them against the *bad-case* time. That made 8 shipments impossible by accident. I fixed it by using the same simulation for both.
- **False confidence on near-ties.** Because scores are rescaled from 0 to 1, two routes only 1% apart could look far apart, and the tool labelled them "High confidence". It now also compares the raw numbers and never calls a near-tie High.

- **Unfair comparison.** My first balance measure compared each choice with routes that broke the rules, which penalises a strategy for not booking an illegal option. It now compares only with routes the operator is actually allowed to book.

7. Assumptions and limitations

- Costs are in US dollars per shipment and time is in door-to-door days. Route figures are estimates, not guarantees.
- Delays on different legs of a route are independent. In reality, a Suez closure delays many routes at once, so I probably underestimate the worst cases.
- A route is only as safe as its riskiest port, so I take the highest risk on the route, not the average. Congestion is averaged across ports.
- Capacity is not modelled: every route can take every shipment.
- Shipment priority and cargo type are given as inputs.
- The data is realistic in shape but not based on real freight prices. The conclusions are about the decision method, not real costs.
- The weights are my judgement. The stability test shows the answers do not depend heavily on them, but they have not been checked with real planners.

8. What I would do next

With real data, I would calibrate prices and delays against real shipment history, especially events where many routes are delayed at once, such as Red Sea disruptions. I would ask real planners to compare pairs of routes and derive the weights from their choices, and I would feed in live port congestion and risk data. I would also add capacity and sailing schedules. An AI model could reword the explanations more naturally, but only if it were not allowed to change any number.

How to check this work: the repository has 92 automated tests, which run on every update. Running `make all` regenerates the data, results and charts. All numbers in this report come directly from that run.